

第7回日本情報オリンピック JOI 2007/2008

# 予選 解説

2007年12月16日・情報オリンピック日本委員会

第7回日本情報オリンピック 予選

## 問題 1 おつり (Change)

配点 20点

競技時間 180分

採点用入力 5組

## 解説

この問題は、肩慣らしとJOI予選の出題形式に慣れてもらうことを意図して出題されている。

この問題で扱われている硬貨は日本と同種類で、1円、5円、10円、50円、100円、500円となっている。今回はおつりの枚数を要求しているので入力した金額を1000から引く必要があることに注意したい。

この問題の硬貨は任意の2枚を選んだ時に小さい硬貨の金額で高い硬貨の金額を割り切ることができるので、単純に高い硬貨からおつりを割り商をおつりの枚数として加算し、そのあまりを次に高い硬貨の金額で割り商をおつりの枚数として加算し、そのあまりをと繰り返していけばよい。

この問題では硬貨の種類は変わらないので、最低限の変数の扱い方と数値の入出力さえわかれば解くことができるだろう。

原文：[https://www2.ioi-jp.org/joi/2007/2008-yo-prob\\_and\\_sol/2008-yo-t1/review/2008-yo-t1-review.html](https://www2.ioi-jp.org/joi/2007/2008-yo-prob_and_sol/2008-yo-t1/review/2008-yo-t1-review.html)

## 問題 2 JOI と IOI (JOI and IOI)

配点 20点

競技時間 180分

採点用入力 5組

### 解説

---

文字列を扱うきわめて基本的な問題である．この問題は2番ということもありプログラミングの基本を問う問題となっている．

まずは問題文をよく読み，入力が最大10000文字の文字列であることを確認する．10000文字の文字列の扱いは，使用するプログラミング言語により，

- 一つの変数に文字列として記憶
- 10000要素の配列を用意し，1文字ずつばらして記憶

または

- 1文字ずつ入力から読み込みながら処理（文字列の各文字に対してランダムアクセスが必要ない場合）

のいずれかになるだろう．この問題の場合には，上記のどの方法を取っても問題はない．

この問題では，与えられた文字列の中に含まれている JOI と IOI の個数を数え上げるだけなので，文字列の先頭より1文字ずつずらしながら連続する3文字をチェックしていき，それぞれの出現数をカウントしていくだけで十分である．1文字ずつ読み込みながら処理する場合には，直前に読み込んだ2文字を記憶しておくだけで事足りる．

どのような方法をとるにせよ，入力文字数を  $n$  とすると  $n$  に比例した時間  $O(n)$  だけで処理できるだろう．

## 問題 2 JOI と IOI (JOI and IOI) 続き

### 解説 (続き)

---

また、採点用データセットの5つの入力データは以下の方針に従いランダムに生成された文字列を使用した。

1. JOIのみ 長さ $180 + \text{JOI}(10\%)$
2. IOIのみ 長さ $180 + \text{IOI}(10\%)$
3. JOI,IOI混合、長さ $180 + \text{JOI}(5\%) + \text{IOI}(5\%)$
4. 混合IOIOIを含む長さ $1000 + \text{JOI}(5\%) + \text{IOI}(5\%) + \text{IOIOI}(1\%)$
5. 混合JOIOIも含む長さ $5000 + \text{JOI}(5\%) + \text{IOI}(5\%) + \text{IOIOI}(1\%) + \text{JOIOI}(1\%)$

特にデータ4と5では、1000文字を超す長い文字列に対応できなかったと思われる者や、この2つにのみ含まれているIOIOIやJOIOIを正しく扱えていない者が一部見受けられたが、この2つのパターンは問題文の入出力データ例にも含まれていたので注意が必要だっただろう。

---

原文：[https://www2.ioi-jp.org/joi/2007/2008-yo-prob\\_and\\_sol/2008-yo-t2/review/2008-yo-t2-review.html](https://www2.ioi-jp.org/joi/2007/2008-yo-prob_and_sol/2008-yo-t2/review/2008-yo-t2-review.html)

## 問題 3 カードゲーム (Card Game)

配点 20点

競技時間 180分

採点用入力 5組

### 解説

---

太郎と花子の2人が、どのようなカードを持っているかのデータを、保存する必要がある。たとえば、サイズ  $2n$  の整数型の配列 `tarou`, `hanako` を確保し、値  $i$  のカードを太郎が持っているなら `tarou[i]=1`, `hanako[i]=0`, 持っていないなら `tarou[i]=0`, `hanako[i]=1` のようにすればよい。

そして、以下のことを、太郎と花子に対して交互に繰り返せばよい。ここで、場に出ているカードに書かれている値が `max` であるとする。また、太郎と花子の手持ちのカードの枚数を、それぞれ保存しておく。

- 手持ちのカードから、`max` よりも大きい値でかつ  $2n$  以下の値のカードのうち、一番小さい値のカードを探す。
  - もしあったなら、持っているカードを保存している配列のその要素を 0 にする。`max` の値を、そのカードの値にする。また、手持ちのカードの枚数を 1 減らす。

自分の持っているカードの枚数が0になったなら、1行目に花子の手持ちのカードの枚数を、2行目に太郎の手持ちのカードの枚数を出力し、終了する。

- もしなかったなら、`max` の値を 0 にする。

---

原文 : [https://www2.ioi-jp.org/joi/2007/2008-yo-prob\\_and\\_sol/2008-yo-t3/review/2008-yo-t3-review.html](https://www2.ioi-jp.org/joi/2007/2008-yo-prob_and_sol/2008-yo-t3/review/2008-yo-t3-review.html)

## 問題 4 星座探し (Searching Constellation)

配点 20点

競技時間 180分

採点用入力 5組

### 解説

反復を用いた基本的な問題である。どのような繰り返しを用いるかという点で少しだけ工夫を要するかもしれない。

基本的には、すべての平行移動の仕方を一つずつ試して、一致する個所を見つけたら出力すればよい。ただし、「すべての平行移動の仕方」は無限にあるので文字通りにすべてを試すのは不可能である。探すべき星座の中の任意の1点を選んで特別扱いし、この点が写真の中の  $n$  個の星の一つに一致するように平行移動する方法をすべて考えれば、平行移動の仕方として高々  $n$  通り試せばよいことがわかる。

$n$  通りの平行移動の仕方を一つずつ試すと決まれば、実際に必要なことは、平行移動後の星座のすべての点が写真に含まれるかどうかを調べることである。含まれていれば平行移動した量を出力して終了し、含まれていなければ次の平行移動の仕方を試せばよい。

素朴に写真の星の座標を与えられたまま長さ  $n$  の配列で記憶した場合、ある点が写真に含まれるかどうかを調べるのに  $O(n)$  時間かかるので、星座全体が含まれるかどうかを調べるには  $O(mn)$  時間かかる。平行移動の仕方は  $O(n)$  通りあるので、全部で  $O(mn^2)$  時間かかることになる。この方法で、各入力に対し高々10秒程度プログラムを実行すれば解ける。

予選では実行時間に制限がないので、この方法で満点を取るのは難しいはずである。

## 問題 4 星座探し (Searching Constellation) 続き

### 解説 (続き)

#### 発展

少し工夫することで、上に述べた方法より高速なアルゴリズムを作ることができる。予選のこの問題では、このような工夫は要求していないが、実際にプログラムを書いて利用しようと思ったら、状況によっては高速化を求められる場合もあるだろう。興味のある人のために、以下では高速化のための方法の概略を述べる。

写真の星の座標を単に与えられたまま配列で記憶するのではなく、この部分の記憶方法 (データ構造) を工夫しておくことで、後で星座のすべての点が写真に含まれるかどうかを調べるのにかかる時間を短縮するのが有効である。このように、後の処理の効率化のために前もって工夫しておくことを前処理という。

写真の星の座標を辞書式順序でソートして覚えておくと、ある点が写真に含まれるかどうかを調べるのに二分探索を用いることができ、 $O(n)$  時間ではなく  $O(\log n)$  時間で済む。このため、全体の計算量は  $O(mn^2)$  時間から  $O(mn \log n)$  時間に改善する。

また、写真の星の座標をハッシュテーブルを用いて記憶しておくと、ある点が写真に含まれるかどうかを調べるのは、運が悪くない限り定数時間で済む。このため、全体の計算量は  $O(mn)$  時間に改善する。

なお、ある点が写真に含まれるかどうかを座標を添え字とする2次元配列で記憶しておけば、ある点が写真に含まれるかどうかの判定は定数時間で行えるが、座標の範囲が  $x$  座標・ $y$  座標とも 0 以上 1000000 以下と広いので、この方法は現実的ではない。この方法でも部分点が取れるよう、入力 1 と入力 2 は座標の範囲を 0 以上 1000 以下に制限してある。

原文 : [https://www2.ioi-jp.org/joi/2007/2008-yo-prob\\_and\\_sol/2008-yo-t4/review/2008-yo-t4-review.html](https://www2.ioi-jp.org/joi/2007/2008-yo-prob_and_sol/2008-yo-t4/review/2008-yo-t4-review.html)

## 問題 5 おせんべい (Rice Crackers)

配点 20点

競技時間 180分

採点用入力 5組

### 解説

この問いでは、最初に横の何行かを同時にひっくり返し、次に縦の何列かを同時にひっくり返すようにと、手順が指定されている。しかし、もし手順が指定されていないとしても  $i$  行  $j$  列の煎餅の状態は、初期状態と第  $i$  行をひっくり返した回数と第  $j$  列をひっくり返した回数だけで定まり、ひっくり返す順序は影響しない。実際、初期状態を  $s$  (表の場合は 1, 裏の場合は 0), 第  $i$  行をひっくり返した回数を  $r$ , 第  $j$  列をひっくり返した回数を  $c$  とすると、 $(s + r + c)$  が偶数ならば煎餅は表で、奇数ならば裏である。

### 解法1

煎餅を焼く機械の縦を  $R$  行, 横を  $C$  列とする。ひっくり返し方は単純に数えると  $2^R \times 2^C$  通りある。これらのひっくり返し方を全て調べれば出荷できる煎餅の枚数の最大値を求めることができる。この方法では、1つ目の採点用テストデータに対して正解できる。2つ目の採点用テストデータは8行50列であるので、調べるひっくり返し方は  $2^8 \times 2^{50} > 10^{17}$  となり、短時間で正解を得ることは難しい。

## 問題 5 おせんべい (Rice Crackers) 続き

### 解説 (続き)

#### 解法2

行のひっくり返し方を1つに固定すると、ある列をひっくり返すかどうかは他の列に影響を及ぼさない、つまり、列ごとに出荷できる煎餅の枚数を最大化すれば、全体の出荷できる煎餅の枚数を最大化したことになる。各列において出荷できる煎餅の枚数を最大化するには、その列の表（と裏）の数を数え、表が多ければひっくり返し、裏が多ければそのままにすればよい。全ての行のひっくり返し方に対して、この方法で出荷できる煎餅の最大枚数を求める。行のひっくり返し方は高々  $2^{10} = 1024$  通りしかないので、適切に実装すると、全ての採点用テストデータに対して正解できる。

#### 解法2の実装方針例

行のひっくり返し方は0と1からなる長さ  $R$  の文字列で表すことができる。ここでは、この文字列の第  $i$  番目のビットが1なら  $i$  行目をひっくり返すことを表し、0なら  $i$  行目はひっくり返さないことを表すことにする。各列の初期状態も、行のひっくり返し方と同様に、0と1からなる長さ  $R$  の文字列で表すことができる。

行のひっくり返し方を上記の方式で表すと  $R$  ビットの非負整数  $i$  になったとする。このひっくり返し方に対して、 $j$  列目の出荷できる煎餅の最大枚数は、 $j$  列目の初期状態を表す  $R$  ビットの非負整数を  $S_j$  とすると、 $i$  と  $S_j$  をビット単位で排他的論理和 (exclusive OR) を取り得られるビット列中の1の数と0の数の最大値  $M_{i,j}$  となる。1から  $C$  までの  $j$  に対して、 $M_{i,j}$  の総和  $M_i$  を求めると、 $M_i$  が行のひっくり返し方  $i$  に対する出荷できる煎餅の最大数となる。

よって、0から  $2^R - 1$  までの各整数  $i$  に対して、 $M_i$  を計算し、これらの最大値が求める数となる。

原文 : [https://www2.ioi-jp.org/joi/2007/2008-yo-prob\\_and\\_sol/2008-yo-t5/review/2008-yo-t5-review.html](https://www2.ioi-jp.org/joi/2007/2008-yo-prob_and_sol/2008-yo-t5/review/2008-yo-t5-review.html)



## 問題 6 船旅 (Boat Travel)

配点 20点

競技時間 180分

採点用入力 5組

### 解説

船舶が次々と運行を開始していく状況の中で、指定された二点間の現在の最小運賃を計算する問題である。

現在までの二点間の最小運賃を格納した二次元配列（例えば、二次元配列 `fares` は、`fares[a][b]` の中に、島 `a` から島 `b` へ向かうときの最小運賃を表す値を格納する）を用いて、新たに運航を開始した船舶の運航情報を受け取るごとにこの二次元配列を更新し、客からの注文票にはこの二次元配列を参照し最小運賃を出力する。

#### 二次元配列の初期化

入力を読み込む前に、船舶が一隻も運航していないときの状態を表した現在までの二点間の最小運賃を表す二次元配列を生成する。

二次元配列の添え字が `i == i` のときは、同じ島であるので一隻も船舶に乗らずに島 `i` から島 `i` へ移動できるため 0 を格納する。

その他のときは、船舶は一隻も運行を開始していないので、別の島に移動することができないため、移動できないことを表した十分に大きな値を格納する。この十分に大きな値とは、（運賃の最大値×島数）よりも十分に大きく、3倍してもオーバーフローしない程度に小さい値とする。十分に大きな値にこのような上限を設けるのは、以下で述べる現在までの二点間の最小運賃を格納した二次元配列を更新する際に、オーバーフローを起こさないようにするためである。また、もしこの十分に大きな値が最小運賃となった場合、島 `a` から島 `b` へ向かう航路が存在しないものとし、出力するときは -1 を出力する。

## 問題 6 船旅 (Boat Travel) 続き

### 解説 (続き)

#### 各入力に対する処理

##### 新たに運航を開始した船舶の運航情報の場合

新たに運航を開始した船舶の運賃が、その船舶が運航する島  $a$  と島  $b$  を結ぶ航路の最小運賃（現在までの二点間の最小運賃を格納した二次元配列に格納されている値）と比較して、以下のように、高い場合、等しい場合、安い場合に場合分けをして考えると、安い場合のみ現在までの二点間の最小運賃を格納した二次元配列を更新すれば良いことが分かる。

- 島  $a$  と島  $b$  を結ぶ航路の最小運賃よりも高い場合

島  $a$  と島  $b$  の間を移動するときはより安価な航路が存在し、島  $a$  と島  $b$  の間を経由するような航路の場合もより安価な航路を使えばよいので、最小運賃にこの運行情報は影響を与えない。

- 島  $a$  と島  $b$  を結ぶ航路の最小運賃と等しい場合

島  $a$  と島  $b$  の間を移動するときは同一価格の航路がすでに存在し、島  $a$  と島  $b$  の間を経由するような航路の場合も最小運賃は変わらないため、最小運賃にこの運行情報は新たな影響を与えない。

- 島  $a$  と島  $b$  を結ぶ航路の最小運賃よりも安い場合

少なくとも島  $a$  と島  $b$  の間を移動するときはこの新たに運行を開始した船舶に乗船することがもっとも安価な移動方法である。また、ある島  $c$  からある島  $d$  へ移動する場合も、この船舶を経由して移動することがもっとも安価となる場合もある。以下のように、新たに運行を開始した船舶に乗船した場合の最小運賃の二通りと乗船しなかった場合の最小運賃の合計三通りに場合分けをして、その中の最小値を計算することによって、新たに一件の運行情報を受け取った場合の二点間の現在までの最小運賃を、計算することができる。

## 問題 6 船旅 (Boat Travel) 続き

### 解説 (続き)

- 出発地から島  $a$  まで最小運賃で向かい、新たに運航を開始した船舶に乗船し島  $a$  から島  $b$  へ向かい、島  $b$  から目的地まで最小運賃で向かう場合の合計運賃。
- 出発地から島  $b$  まで最小運賃で向かい、新たに運航を開始した船舶に乗船し島  $b$  から島  $a$  に向かい、島  $a$  から目的地まで最小運賃で向かう場合の合計運賃。
- 新たに運航を開始した船舶を使わず、目的地から出発地へ向かう場合の最小運賃。つまり、更新前の現在までの二点間の最小運賃を保存している二次元配列に格納されている値。

二点間の現在までの最小運賃を格納した二次元配列の出発地と目的地の任意の組み合わせは、以上の三通りの中の最小値に更新すればよい。

また、新たに運行を開始した船舶の運行情報一件あたりの現在までの二点間の最小運賃を保存している二次元配列を更新するための計算量は、島の数  $n$  とおくと、 $O(n^2)$  である。

#### 客からの注文票の場合

現在までの二点間の最小運賃を格納している二次元配列を参照し、出力する。

二次元配列に格納されている島  $a$  から島  $b$  への最小運賃の値が十分に大きな数であった場合、島  $a$  から島  $b$  に移動する航路が存在していないことを表しているため、-1 を出力する。それ以外の場合は、この二次元配列に格納されている値を出力する。

よって、出力に要する計算量は  $O(1)$  である。

以上から、全体の計算量は、船舶の運航開始情報の数を  $k$  とおくと、 $O(k n^2)$  である。

#### 別解法

各島を頂点、船舶を辺、船舶の運賃を辺の重みとした重み付き無向グラフを構築すると、重み付きグラフに対する最短経路を求めるアルゴリズムを使うことによって、この問題を解くことができる。

## 問題 6 船旅 (Boat Travel) 続き

### 解説 (続き)

最短経路を求めるアルゴリズムには、DijkstraのアルゴリズムやFroid-Worshalのアルゴリズムなどがある。

Dijkstraのアルゴリズムの場合の計算量は、新たに運航を開始した船舶の運航情報の場合は一件あたり  $O(1)$  であり、客からの注文票の場合は一件あたり  $O((k + n) \log n)$  である。よって、全体の計算量は、客からの注文票の数を  $m$  とおくと、 $O(m(k + n) \log n)$  である。

Froid-Worshalのアルゴリズムの場合の計算量は、船舶の運行情報一件あたりの計算量は  $O(n^3)$  であり、客からの注文票で最小運賃を計算する場合は  $O(1)$  である。よって、全体の計算量は、 $O(kn^3)$  である。

入力データの大きさから三つの解法の中でもっとも時間のかかるFroid-Worshalのアルゴリズムを用いた場合であっても、各入力に対し高々10秒程度プログラムを実行すれば解ける。

注：公式解説では「Froid-Worshal」と表記されています。一般的な綴りは「Floyd-Warshall」ですが、本文では原文の表記を保持しています。

原文：[https://www2.ioi-jp.org/joi/2007/2008-yo-prob\\_and\\_sol/2008-yo-t6/review/2008-yo-t6-review.html](https://www2.ioi-jp.org/joi/2007/2008-yo-prob_and_sol/2008-yo-t6/review/2008-yo-t6-review.html)

出典：情報オリンピック日本委員会「第7回日本情報オリンピック JOI 2007/2008 予選競技課題」。原資料は CC BY-SA 4.0 で提供されています。本PDFは各解説ページを問題順に再構成したものです。  
[https://www2.ioi-jp.org/joi/2007/2008-yo-prob\\_and\\_sol/index.html](https://www2.ioi-jp.org/joi/2007/2008-yo-prob_and_sol/index.html)